

HTML & CSS

Frontend Development

Semantic HTML in Practice

Semantic HTML means picking tags for what content is, not how it looks. A page built entirely from `<div>` tags might look identical to one using `<header>`, `<nav>`, `<main>`, `<article>`, and `<footer>` — but the semantic version is accessible to screen readers, easier for search engines to index correctly, and easier for another developer to understand at a glance. A useful rule: before reaching for a generic `<div>`, check whether a more specific tag already exists for what you're building (a list is `<ul>/<ol>`, a form control's label uses `<label for=...>`, a button that submits a form is `<button>`, not a styled `<div>` with a click handler).

Forms and Accessibility

Forms are one of the most common places accessibility gets ignored. Every input should have an associated `<label>` (linked via the `for/id` attributes, or wrapping the input) so screen readers announce what the field is for, and so clicking the label focuses the input. Required fields should use the `required` attribute (which also gives free browser-level validation), and error messages should be associated with their field via `aria-describedby` so assistive technology announces them. Getting forms right isn't just about compliance — it directly reduces how many users give up partway through filling one out.

The CSS Box Model, Precisely

Every element's rendered size is the sum of its content, padding, border, and margin. By default, width/height apply only to the content box, so adding padding or a border increases the element's total rendered size beyond what you set — a frequent source of layout bugs. Setting `box-sizing: border-box` (commonly applied globally to all elements) changes this so width/height include padding and border, making sizing far more predictable and matching how most developers actually think about element size.

Flexbox in Depth

Flexbox distributes space along one axis at a time.

- `justify-content` controls alignment along the main axis (e.g. horizontally in a row layout) — `flex-start`, `center`, `space-between`, and `space-around` are the ones used most often.
- `align-items` controls alignment along the cross axis (e.g. vertically in a row layout) — `stretch` (default), `center`, `flex-start`, `flex-end`.
- `flex-grow`/`flex-shrink`/`flex-basis` (usually shorthand as `flex: 1`) control how individual items grow or shrink to fill available space.

- flex-wrap lets items wrap onto multiple lines instead of overflowing or shrinking indefinitely.

CSS Grid in Depth

Grid defines both rows and columns explicitly, then places items into that structure. grid-template-columns/grid-template-rows define the track sizes (fixed values, percentages, or the flexible fr unit, which divides remaining space proportionally). Named grid areas (grid-template-areas) let you lay out a page visually in the CSS itself, mapping named regions like 'header', 'sidebar', and 'main' to a readable ASCII-art-style grid — often the clearest way to express a full page layout, especially one that needs to rearrange at different breakpoints.

Responsive Design Principles

Mobile-first responsive design means writing default styles for the smallest expected screen, then layering on complexity with min-width media queries as the viewport grows — rather than writing for desktop first and trying to compress it down. Common breakpoints (though every project should base them on its actual content, not fixed numbers) roughly separate mobile, tablet, and desktop layouts. Beyond media queries, relative units (rem for text so it respects user font-size preferences, % or fr for flexible widths, vw/vh sparingly for full-viewport elements) do most of the work of making a layout naturally adapt without needing a breakpoint for every small size change.

Learn More

MDN Web Docs — HTML & CSS Reference

<https://developer.mozilla.org/en-US/docs/Web>

CSS-Tricks — A Complete Guide to Flexbox

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

web.dev — Learn Responsive Design

<https://web.dev/learn/design/>